

Pandas

Reading Data

Dario Pescini

Università degli Studi di Milano-Bicocca

Department of Statistics and Quantitative Methods

dario.pescini@unimib.it

What is Pandas?

Pandas is the standard Python library for working with tabular data.

Two data structures

- **Series** — one-dimensional labelled array
- **DataFrame** — two-dimensional table (rows $\times$ columns)

Two core tasks

- **I/O** — reading and writing many file formats
- **Manipulation** — filtering, reshaping, aggregating

Import convention used everywhere:

```
import pandas as pd
```

Series

A **Series** is an ordered sequence of values with a labelled **index**.

```
apples = pd.Series([3, 2, 0, 1], name="Apples")  
print(apples)
```

```
0    3  
1    2  
2    0  
3    1
```

```
Name: Apples, dtype: int64
```

The index can be any comparable type:

```
apples = pd.Series([3, 2, 0, 1], name="Apples",  
                    index=['Mon', 'Tue', 'Wed', 'Thu'])
```

DataFrame

A **DataFrame** is a table: columns are **Series** that share the same **index**.

```
fruit = pd.DataFrame({  
    'Apples': [3, 2, 0, 1],  
    'Oranges': [0, 3, 7, 2]  
})
```

	Apples	Oranges
0	3	0
1	2	3
2	0	7
3	1	2

In practice we almost never create DataFrames by hand — we **read them from files**.

File Formats

Format	Extension	Pandas function
Comma-separated values	.csv	<code>pd.read_csv()</code>
Excel workbook	.xls/.xlsx	<code>pd.read_excel()</code>
JavaScript Object Notation	.json	<code>pd.read_json()</code>

All three functions return a **DataFrame** and accept a local path or a URL.

Focus of this lecture

CSV is the dominant format in computational biology (sequencing pipelines, expression matrices, metadata tables). The lecture covers `read_csv` in depth. `read_excel` and `read_json` work analogously — just swap the function name.

Reading Excel — `pd.read_excel()`

```
labor = pd.read_excel("RawData/CLF01.xls")
```

For files with multiple sheets, name the one you want:

```
labor = pd.read_excel("RawData/CLF01.xls",  
                      sheet_name="Sheet2")
```

Dependency

Requires the `openpyxl` package. Install once:

```
conda install openpyxl
```

Reading CSV — Simplest Case

`pd.read_csv()` assumes comma separator and a header row:

```
meteo = pd.read_csv("RawData/MeteoMilano2011.csv")  
print(meteo.shape)
```

```
(365, 23)
```

```
print(meteo.columns.tolist()[:4])
```

```
['CET', 'Temperatura maxC', 'Temperatura mediaC',  
 'Temperatura minC']
```

Dataset: daily weather in Milan 2011. One row per day, 23 measured variables.

CSV: No Header — names=

Some files have no header line at all. Without `names=`, the first data row becomes the header:

```
# Wrong: first row treated as column names
nani = pd.read_csv("RawData/7-nani.csv")
```

```
# Correct: supply column names explicitly
nani = pd.read_csv("RawData/7-nani.csv",
                   names=['Name'])
```

```
      Name
0  Grumpy
1  Sleepy
2  Bashful
...
```


CSV: Custom Separator — delimiter=

Italian open-data files commonly use **semicolons** as separators (to avoid ambiguity with the decimal comma):

```
iscritti = pd.read_csv(
    "RawData/2009-2013_iscritti.csv",
    delimiter=';'
)
iscritti.head(2)
```

	ANNO_ACCADEMICO	COD_ATENEO	NOME_ATENEO	...
0	2008-2009	00101	Torino - Università	...
1	2008-2009	00102	Torino - Politec	...

delimiter and sep are interchangeable.

CSV: European Number Format

Continental European convention swaps comma and dot:

1.234.567,89 instead of 1,234,567.89

```
gettito = pd.read_csv(  
    "RawData/2009-2013_gettito_contribuzione.csv",  
    delimiter=';',  
    decimal=','  
)
```

Without `decimal=','` the CONSUNTIVO column is loaded as a **string** instead of a number — all arithmetic will silently fail.

Dates: The Golden Rule

Rule

Always convert date columns **at load time**, inside `read_csv()`.
Never leave them as strings and convert later.

Why it matters:

- A string column silently passes through arithmetic and groupby – producing wrong results without any error
- Post-hoc conversion is easy to forget, especially in multi-step pipelines
- `parse_dates=` documents the intent: *this column is a date*

Priority

1. `parse_dates=` in `read_csv()` (always try this first)
2. `pd.to_datetime()` after loading (fallback only)

Dates: parse_dates= at Load Time

Pass the list of date column names directly to `read_csv()`:

```
struttire = pd.read_csv(
    "RawData/2009-2013_struttire.csv",
    delimiter      = ';',
    parse_dates    = ['DATA'],      # name, not index
    dayfirst       = True           # DD/MM/YY European format
)
struttire.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: ... entries
#   Column   Non-Null Count  Dtype
0   DATA    ...           datetime64[ns]    <- correct
1   ...
```

`dayfirst=True` is required whenever the day precedes the month (01/11/09).

Prefer column **names** over integer indices — more readable, robust to column reordering.

Dates: Always Inspect Visually

Even when `parse_dates` succeeds, **compare parsed values against the raw file**.

Silent failures produce wrong dates, not errors:

```
# Raw strings (reload without parse_dates)
raw = pd.read_csv("RawData/2009-2013_struttire.csv",
                  delimiter=';', nrows=3)
print(raw.iloc[:, 0].values)      # e.g. ['01/11/09' ...]

# Parsed result
print(struttire['DATA'].head(3))  # should be 2009-11-01
```

Classic silent failures

- Day and month swapped — missing `dayfirst=True`
- Wrong century — 2-digit year parsed as 1923 instead of 2023
- Sentinel strings (e.g. 9999-12-31) silently become NaT

Dates: Fallback — `pd.to_datetime()`

If `parse_dates` cannot handle the format, use `pd.to_datetime()` as a **second step**.

Specify the exact format with `format=`:

```
# parse_dates alone failed – column is still object  
struttura['DATA'] = pd.to_datetime(  
    struttura['DATA'], format="%d/%m/%y"  
)
```

Code	Example	Meaning
%d	25	day (01–31)
%m	03	month (01–12)
%Y	2023	4-digit year
%y	23	2-digit year*

*%y: 00–68 → 2000–2068; 69–99 → 1969–1999

After converting, repeat the visual inspection and range check.

Dates: Handling Bad Values — `errors='coerce'`

By default `pd.to_datetime()` **raises** an exception on the first unparseable value.

`errors='coerce'` replaces bad values with `NaT` instead:

```
s = pd.Series(["01/03/2023", "not-a-date", "15/06/2022"])
pd.to_datetime(s, format="%d/%m/%Y", errors='coerce')
```

```
0    2023-03-01
1           NaT
2    2022-06-15
dtype: datetime64[ns]
```

Then verify how many values were lost:

```
print(struttura['DATA'].isna().sum())    # unexpected NaT?
print(struttura['DATA'].min(), struttura['DATA'].max())
```

If `min()` or `max()` returns `NaT`, the format string does not match the file.

Dates: The .dt Accessor

A `datetime64` column exposes the `.dt` accessor for extracting components:

```
struttura['year']      = struttura['DATA'].dt.year
struttura['month']     = struttura['DATA'].dt.month
struttura['day']       = struttura['DATA'].dt.day
struttura['weekday']  = struttura['DATA'].dt.day_name( )
```

Useful for grouping or filtering by time components:

```
struttura[struttura['DATA'].dt.year == 2013]
struttura.groupby(struttura['DATA'].dt.month).size( )
```

Note

`.dt` is available only on `datetime64` columns. Calling it on an object column raises `AttributeError` — a reliable sign that the conversion did not work.

CSV: Missing Values — `na_values=`

Pandas recognises several strings as NaN by default (`"", NA, NaN, null, ...`).

For domain-specific sentinel values, add them:

```
iscritti = pd.read_csv(
    "RawData/2009-2013_iscritti.csv",
    delimiter=';',
    na_values=['n.d.', '---']
)
```

To disable the defaults and use only your list:

```
pd.read_csv("file.csv",
            keep_default_na=False,
            na_values=['n.d.'])
```

CSV: Character Encoding — encoding=

Files produced on Windows often use **Latin-1**:

```
spesa = pd.read_csv(  
    "RawData/2009-2013_spesa_interventi.csv",  
    delimiter=';',  
    encoding='iso-8859-1'  
)
```

Common encodings

- utf-8 — default, universal
- iso-8859-1 — Western Europe
- cp1252 — Windows Western Europe

Symptom

UnicodeDecodeError
when calling read_csv

CSV: Large Files

For files with millions of rows, inspect a sample first:

```
# Read only the first 100 rows
sample = pd.read_csv("RawData/large_file.csv",
                     nrows=100)
```

```
# Suppress dtype-inference warnings
df = pd.read_csv("RawData/large_file.csv",
                 low_memory=False)
```

- `nrows=N`: read only the first N rows — fast and memory-safe
- `low_memory=False`: scan the entire file before inferring column types

CSV: Setting the Index — `index_col=`

Use an existing column as the row label instead of the default 0, 1, 2, ...:

```
iscritti = pd.read_csv(
    "RawData/2009-2013_iscritti.csv",
    delimiter=';',
    index_col='COD_ATENEO'
)
iscritti.head(2)
```

	ANNO_ACCADEMICO	NOME_ATENEO ...
COD_ATENEO		
00101	2008-2009	Torino - Università ...
00102	2008-2009	Torino - Politec ...

Putting It All Together

Real files combine **multiple** format challenges simultaneously:

```
gettito = pd.read_csv(
    "RawData/2009-2013_gettito_contribuzione.csv",
    delimiter=';',          # Italian CSV
    decimal=',',           # European numbers
    na_values=['n.d.'],     # domain missing values
    index_col='COD_Ateneo'  # row labels
)
```

Debugging workflow

Start with `nrows=5` to inspect the raw content. Add parameters one at a time until the file loads correctly.

Parameter Reference

Parameter	Purpose
<code>names=</code>	column names (when there is no header)
<code>delimiter=</code>	field separator (default: <code>' , '</code>)
<code>decimal=</code>	decimal-point character (default: <code>' .' '</code>)
<code>parse_dates=</code>	columns to convert to datetime
<code>dayfirst=</code>	interpret DD/MM/YY date format
<code>na_values=</code>	additional missing-value tokens
<code>keep_default_na=</code>	disable built-in missing-value list
<code>encoding=</code>	character encoding of the file
<code>nrows=</code>	read only the first N rows
<code>low_memory=</code>	scan all rows before inferring types
<code>index_col=</code>	column to use as row index